

Symbolica Integration

Symbolic integration with genuine Rubi steps

Symbolic integration within Symbolica: compute an antiderivative or inspect the complete nested Rubi rule trace.

The official integration plugin, powered by Symbolica 3.0. Free for any use within Typst. No license or license key needed.

Repository · MIT license

1 Use Rubi from Symbolica

Symbolica includes Rubi integration alongside parsing, algebra, differentiation, and rendering in one package and one WebAssembly engine. Integration accepts the same expression values and symbol handles as the rest of the API.

It is free to use for any use within Typst, including academic and commercial work. No Symbolica license or license key is needed.

Import Symbolica once to use algebra and integration together:

```
#import "@preview/symbolica:0.1.0" as sym

#let x = sym.math($x$)
#let f = sym.math($x / (x + 1)$)
#let primitive = sym.integrate(f, x)

$ integral #sym.to-tyrst(f) dif x = #sym.to-tyrst(primitive) + C $
```

The first call to `integrate` or `integrate-with-steps` compiles and initializes the integration rule sets, which may take about 10 seconds. Both functions share the cached rules, so later calls and ordinary edits reuse them. Restarting the compiler or clearing its cache repeats this setup.

1.1 A worked rule trace

$$\int x(1+x)^{-1} dx = x - \log(1+x) + C$$

Rule 49 : Algebraic expansion

$$\int x(1+x)^{-1} dx = \int 1 - (1+x)^{-1} dx$$

Rule 2009 : Integrate the terms of the sum separately.

$$\int 1 - (1+x)^{-1} dx = \int 1 dx + \int (-1-x)^{-1} dx$$

Rule 24 : Apply the direct antiderivative formula.

$$\int 1 dx = x$$

Rule 16 : Apply the direct antiderivative formula.

$$\int -1(1+x)^{-1} dx = -1 \log(1+x)$$

`complete` is true when Rubi eliminated every integral. A partial result is still returned when it is false. No integration constant is added. Symbol namespaces, attributes, and portable attachments are retained in the result and in each step payload.

2 Public API

This guide covers the two integration functions in the Symbolica API.

Function	Result
<code>integrate(expression, variable)</code>	The best-effort antiderivative as portable Atom payload bytes.
<code>integrate-with-steps(expression, variable)</code>	A dictionary containing result, complete, and nested steps.

The expression may be an Atom payload or another supported expression value. The integration variable must represent one symbol. Both `math($x$)` and `symbol("x")` are accepted directly. Use `init(namespace: "model")` when your integration and other algebra operations need a shared custom namespace.

2.1 Step fields

Each step contains:

- `rule`: the Rubi rule number, or none for an auxiliary transformation;
- `depth`: nesting depth of the rewritten integral;

- description, references, and source: Rubi's explanatory metadata; and
- input and output: portable Atom payload bytes for the immediate rewrite.

The array is ordered from the outer rewrite into nested integrals. Use the `depth` field to indent it, group it, or build a custom explanation layout.

3 Development installation

For development before publication, expose the repository root as one local package and use `@local` in place of `@preview`. On Linux:

```
mkdir -p ~/.local/share/typst/packages/local/symbolica
ln -s /path/to/symbolica-typst-plugin \
    ~/.local/share/typst/packages/local/symbolica/0.1.0
```

The package contains one uncompressed `symbolica.wasm`, loaded directly by Typst. Package downloads use archive compression. Rule tables are constructed on first integration use; algebra-only documents do not prepare them.

Symbolica Integrate uses Symbolica's integration interface and the Rubi ruleset. Thanks to the Symbolica and Rubi contributors for making the engine possible.